FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

PuzzleMind

INTERACTIVE CSP & SEARCH VISUALIZATION SUITE

A PROJECT REPORT

Submitted by

240701626 – Devadharshini K

240701022 – Afrin

RAJALAKSHMI ENGINEERING COLLEGE
(An Autonomous Institution)
Chennai, Tamil Nadu

October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "PuzzleMind: Interactive CSP & Search Visualization Suite" is the bonafide work of "Devadharshini K (240701626) and Afrin (240701022)" who carried out the project work under my supervision.

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER SIGNATURE OF THE FACULTY INCHARGE

TABLE OF CONTENTS

ABSTRACT

1	INTRODUCTION
2	LITERATURE REVIEW
3	PROPOSED SYSTEM
3.1	ARCHITECTURE DIAGRAM
4	MODULES DESCRIPTION
4.1	PUZZLE SELECTION AND BOARD MODULE
4.2	CSP SOLVER MODULE
4.3	FORWARD CHECKING MODULE
4.4	VISUALIZATION AND PLAYBACK MODULE
4.5	STATISTICS MODULE
5	IMPLEMENTATION AND RESULTS
5.1	EXPERIMENTAL SETUP
5.2	RESULTS
6	CONCLUSION AND FUTURE WORK
7	REFERENCES

ABSTRACT

PuzzleMind is an interactive web-based suite designed to help learners understand Constraint Satisfaction Problems (CSPs) and search algorithms through visual, step-by-step execution. The system brings together four puzzle domains: Sudoku, N-Queens, Map Coloring, and Kakuro. Rather than displaying only a final solution, PuzzleMind aims to expose the solver's choices, candidate domains, constraint conflicts, domain pruning, and backtracking behaviour. Its central methods include recursive backtracking, the Minimum Remaining Values (MRV) heuristic, and forward checking. The interface is described as being built with React and Vite, while the solver evaluates constraints and produces visualization events that update the interface. Interactive controls include starting, pausing

stepping through, changing playback speed, and resetting the board. The presentation reports a Sudoku benchmark in which basic backtracking explored 4,209 nodes while MRV explored 52 nodes, with 102 candidate values pruned through forward checking and a reported 98.7% search-tree reduction. These figures are the benchmark results presented in the project slides and should be interpreted in the context of that test case. PuzzleMind's main contribution is educational transparency: it makes the internal search process observable so that learners can connect AI concepts with solver behaviour.

1 INTRODUCTION

a. Background

Constraint Satisfaction Problems (CSPs) are a class of artificial intelligence problems represented using variables, domains of possible values, and constraints that restrict which combinations are valid. Examples include Sudoku, N-Queens, graph or map coloring, and arithmetic logic puzzles. CSPs are useful for explaining how intelligent systems search through alternatives while respecting rules.

b. Problem Statement

Many conventional puzzle solvers focus on presenting the completed board or final answer. This can make it difficult for learners to understand why a value was selected, why a candidate was rejected, how a domain changed, or when and why the solver backtracked. Heuristics may therefore remain abstract concepts rather than observable processes.

c. Objectives

The objectives of PuzzleMind are to provide one interface for multiple puzzle domains; visualize solver decisions step by step; demonstrate MRV-based variable selection and forward checking; expose candidate-domain changes and conflicts; and show metrics such as explored nodes, backtracks, and pruned candidates.

d. Scope

The project focuses on educational visualization and interactive exploration. The presented puzzle suite includes Sudoku, N-Queens, Map Coloring, and Kakuro. The system prioritizes clarity of execution over raw high-throughput solving performance.

2 LITERATURE REVIEW

a. Constraint Satisfaction Problem Model

A CSP is commonly defined by a set of variables, a domain for each variable, and constraints specifying acceptable assignments. A solution is a complete assignment that satisfies all constraints. This formal model provides a common foundation for different puzzle types.

b. Backtracking Search

Backtracking is a depth-first search technique. It assigns a value to an unassigned variable, checks whether the partial assignment is consistent, and continues when valid. If a conflict occurs or no candidate remains, the algorithm reverses an earlier assignment and tries another choice.

c. Minimum Remaining Values Heuristic

The Minimum Remaining Values (MRV) heuristic chooses the unassigned variable with the fewest currently valid values. The intention is to expose difficult choices early and reduce unnecessary exploration of the search tree.

d. Forward Checking and Domain Pruning

Forward checking updates the domains of neighbouring variables after an assignment by removing values that would violate constraints. If a neighbour's domain becomes empty, the current assignment cannot lead to a solution and the solver can backtrack.

e. Identified Gap

The project presentation identifies three limitations in traditional learning experiences: single-puzzle focus, abstract rather than visible heuristic metrics, and opaque execution in which rejection reasons and domain-pruning events are hidden. PuzzleMind addresses these by combining multiple puzzle solvers with interactive visualization. This review summarizes the concepts stated in the project presentation rather than claiming a comprehensive survey of published research.

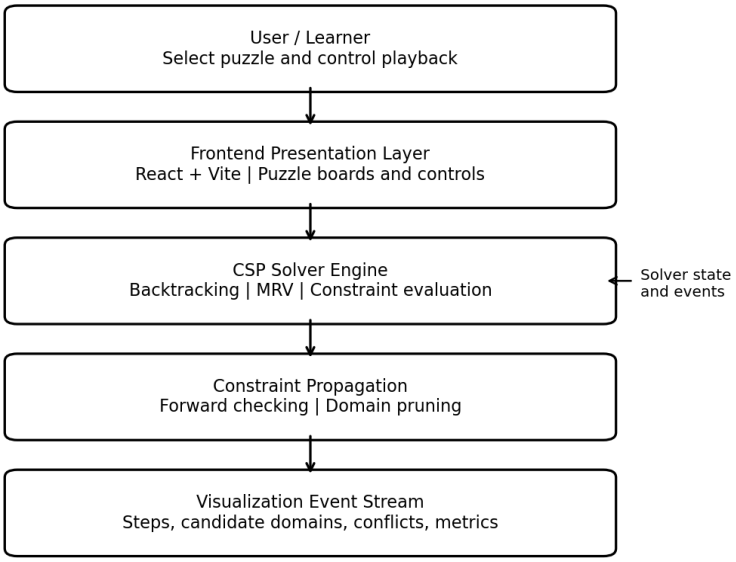
3 PROPOSED SYSTEM

a. Overview

PuzzleMind is proposed as an interactive web application that connects puzzle boards with a CSP solver and a visualization layer. Users select a puzzle and inspect the solver's progress using playback controls. The solver's internal decisions are represented as events that can be rendered by the interface.

b. Architecture Diagram

The architecture presents the learner interface, React/Vite frontend, CSP solver, constraint propagation, and visualization events.



The architecture shown below is derived from the system layers described in the project presentation. The diagram presents the learner interface, React/Vite frontend, CSP solver, constraint propagation, and visualization events.

c. Working Principle

The solver represents the selected puzzle using variables, domains, and constraints. It selects an unassigned variable using MRV, tries a candidate value, checks constraints, and applies forward checking to affected domains. If the choice creates a conflict, it backtracks. The interface displays the resulting state

d. Puzzle Selection and Board Module

This module provides the four puzzle domains identified in the presentation. Sudoku uses a 9×9 grid with row, column, and sub-grid constraints. N-Queens places queens on an N×N board so that no two queens attack each other. Map Coloring assigns colors to graph vertices while ensuring adjacent vertices differ. Kakuro uses arithmetic cross-sum constraints and unique-digit rules within runs.

e. CSP Solver Module

The solver module performs recursive backtracking and evaluates constraints before accepting assignments. It uses MRV to choose the next unassigned variable with the smallest remaining domain. When a candidate assignment is inconsistent, the solver rejects it and explores another candidate.

f. Forward Checking Module

After a variable is assigned, forward checking removes incompatible values from neighbouring variable domains. This domain pruning can detect dead ends earlier than waiting for a later assignment to fail.

g. Visualization and Playback Module

The visualization layer presents candidate-domain changes, active constraints, search depth, conflicts, and solver metrics. The project presentation lists Start, Pause, Step Next, Speed Slider, and Board Reset controls. An event-driven stream updates the React interface as solver decisions occur.

h. Statistics Module

The statistics view is intended to make heuristic behaviour measurable through values such as explored nodes, backtracks, and pruned candidates. These metrics help learners compare basic backtracking with heuristic-guided search.

4 IMPLEMENTATION AND RESULTS

a. Experimental Setup

The presentation identifies React and Vite for the frontend, a recursive CSP solver for search and constraint evaluation, and an event stream for visualization updates. The four implemented puzzle categories are Sudoku, N-Queens, Map Coloring, and Kakuro. The reported quantitative comparison is a Sudoku benchmark. The supplied slides do not specify the hardware, operating system, browser, exact puzzle-generation procedure, or repeated-trial protocol; these details should be added if available.

b. Results

The project presentation reports the following Sudoku benchmark values: basic backtracking explored 4,209 nodes; MRV explored 52 nodes; 102 candidate values were pruned live through forward checking; and the reported search-tree reduction was 98.7%. These results illustrate the potential impact of variable ordering and early domain pruning for the tested puzzle.

c. Result Interpretation

The reduction from 4,209 to 52 explored nodes indicates substantially less search for the benchmark shown in the presentation. Forward checking contributes by eliminating values that cannot remain valid after an assignment. Results may differ for other puzzle instances, so these figures should not be treated as a universal performance guarantee.

d. Limitations

The presentation notes that the system is optimized for clear visualization rather than maximum solving throughput. Large puzzles may generate extensive event logs during playback. The supplied material does not include a detailed evaluation across multiple puzzle difficulties or a formal usability study.

5 CONCLUSION AND FUTURE WORK

a. Conclusion

PuzzleMind connects CSP theory with an interactive visual exploration environment. It brings together four puzzle domains and demonstrates backtracking, MRV variable ordering, forward checking, and domain pruning. Its key educational value is that learners can observe choices, conflicts, backtracks, and candidate-domain states instead of seeing only the final result. The Sudoku benchmark in the presentation reports a 98.7% search-tree reduction for the tested case.

b. Future Work

The project presentation identifies custom user-created puzzle boards, Arc Consistency (AC-3), and benchmark exports as future directions. Additional improvements could include documenting repeatable benchmark conditions, testing a broader set of puzzle instances, and offering downloadable execution traces for classroom analysis. These are recommendations for future development, not features claimed as already implemented.

6 REFERENCES

The following foundational references are relevant to the CSP methods discussed in the project. They are listed alphabetically by first author, following the supplied report-format guidance.

Dechter, R. (2003). Constraint Processing. Morgan Kaufmann.

Russell, S. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach. 4th ed. Pearson.